

We use the *mark* attributes to obtain the desired time bounds. They record a little piece of the history of each node. Suppose that the following events have happened to node x :

1. at some time, x was a root,
2. then x was linked to (made the child of) another node,
3. then two children of x were removed by cuts.

As soon as the second child has been lost, we cut x from its parent, making it a new root. The attribute $x.mark$ is TRUE if steps 1 and 2 have occurred and one child of x has been cut. The CUT procedure, therefore, clears $x.mark$ in line 4, since it performs step 1. (We can now see why line 3 of FIB-HEAP-LINK clears $y.mark$: node y is being linked to another node, and so step 2 is being performed. The next time a child of y is cut, $y.mark$ will be set to TRUE.)

We are not yet done, because x might be the second child cut from its parent y since the time that y was linked to another node. Therefore, line 7 of FIB-HEAP-DECREASE-KEY attempts to perform a **cascading-cut** operation on y . If y is a root, then the test in line 2 of CASCADING-CUT causes the procedure to just return. If y is unmarked, the procedure marks it in line 4, since its first child has just been cut, and returns. If y is marked, however, it has just lost its second child; y is cut in line 5, and CASCADING-CUT calls itself recursively in line 6 on y 's parent z . The CASCADING-CUT procedure recurses its way up the tree until it finds either a root or an unmarked node.

Once all the cascading cuts have occurred, lines 8–9 of FIB-HEAP-DECREASE-KEY finish up by updating $H.min$ if necessary. The only node whose key changed was the node x whose key decreased. Thus, the new minimum node is either the original minimum node or node x .

Figure 19.5 shows the execution of two calls of FIB-HEAP-DECREASE-KEY, starting with the Fibonacci heap shown in Figure 19.5(a). The first call, shown in Figure 19.5(b), involves no cascading cuts. The second call, shown in Figures 19.5(c)–(e), invokes two cascading cuts.

We shall now show that the amortized cost of FIB-HEAP-DECREASE-KEY is only $O(1)$. We start by determining its actual cost. The FIB-HEAP-DECREASE-KEY procedure takes $O(1)$ time, plus the time to perform the cascading cuts. Suppose that a given invocation of FIB-HEAP-DECREASE-KEY results in c calls of CASCADING-CUT (the call made from line 7 of FIB-HEAP-DECREASE-KEY followed by $c - 1$ recursive calls of CASCADING-CUT). Each call of CASCADING-CUT takes $O(1)$ time exclusive of recursive calls. Thus, the actual cost of FIB-HEAP-DECREASE-KEY, including all recursive calls, is $O(c)$.

We next compute the change in potential. Let H denote the Fibonacci heap just prior to the FIB-HEAP-DECREASE-KEY operation. The call to CUT in line 6 of

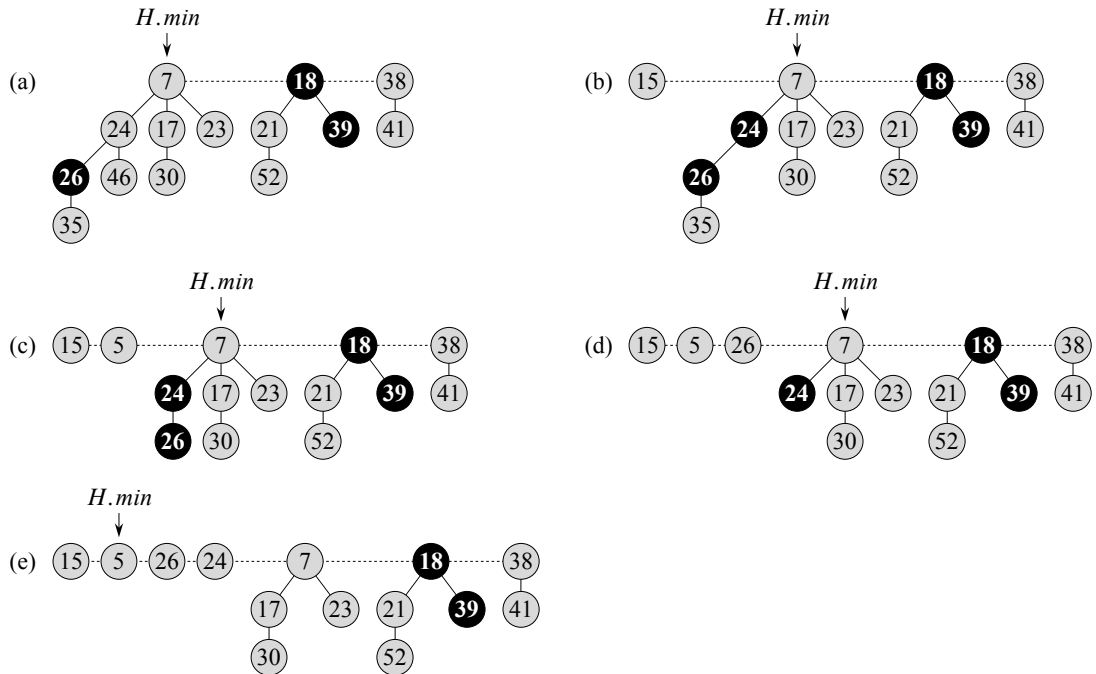


Figure 19.5 Two calls of FIB-HEAP-DECREASE-KEY. **(a)** The initial Fibonacci heap. **(b)** The node with key 46 has its key decreased to 15. The node becomes a root, and its parent (with key 24), which had previously been unmarked, becomes marked. **(c)–(e)** The node with key 35 has its key decreased to 5. In part (c), the node, now with key 5, becomes a root. Its parent, with key 26, is marked, so a cascading cut occurs. The node with key 26 is cut from its parent and made an unmarked root in (d). Another cascading cut occurs, since the node with key 24 is marked as well. This node is cut from its parent and made an unmarked root in part (e). The cascading cuts stop at this point, since the node with key 7 is a root. (Even if this node were not a root, the cascading cuts would stop, since it is unmarked.) Part (e) shows the result of the FIB-HEAP-DECREASE-KEY operation, with $H.min$ pointing to the new minimum node.

FIB-HEAP-DECREASE-KEY creates a new tree rooted at node x and clears x 's mark bit (which may have already been FALSE). Each call of CASCADING-CUT, except for the last one, cuts a marked node and clears the mark bit. Afterward, the Fibonacci heap contains $t(H) + c$ trees (the original $t(H)$ trees, $c - 1$ trees produced by cascading cuts, and the tree rooted at x) and at most $m(H) - c + 2$ marked nodes ($c - 1$ were unmarked by cascading cuts and the last call of CASCADING-CUT may have marked a node). The change in potential is therefore at most

$$((t(H) + c) + 2(m(H) - c + 2)) - (t(H) + 2m(H)) = 4 - c.$$

Thus, the amortized cost of FIB-HEAP-DECREASE-KEY is at most

$$O(c) + 4 - c = O(1) ,$$

since we can scale up the units of potential to dominate the constant hidden in $O(c)$.

You can now see why we defined the potential function to include a term that is twice the number of marked nodes. When a marked node y is cut by a cascading cut, its mark bit is cleared, which reduces the potential by 2. One unit of potential pays for the cut and the clearing of the mark bit, and the other unit compensates for the unit increase in potential due to node y becoming a root.

Deleting a node

The following pseudocode deletes a node from an n -node Fibonacci heap in $O(D(n))$ amortized time. We assume that there is no key value of $-\infty$ currently in the Fibonacci heap.

FIB-HEAP-DELETE(H, x)

- 1 FIB-HEAP-DECREASE-KEY($H, x, -\infty$)
- 2 FIB-HEAP-EXTRACT-MIN(H)

FIB-HEAP-DELETE makes x become the minimum node in the Fibonacci heap by giving it a uniquely small key of $-\infty$. The FIB-HEAP-EXTRACT-MIN procedure then removes node x from the Fibonacci heap. The amortized time of FIB-HEAP-DELETE is the sum of the $O(1)$ amortized time of FIB-HEAP-DECREASE-KEY and the $O(D(n))$ amortized time of FIB-HEAP-EXTRACT-MIN. Since we shall see in Section 19.4 that $D(n) = O(\lg n)$, the amortized time of FIB-HEAP-DELETE is $O(\lg n)$.

Exercises

19.3-1

Suppose that a root x in a Fibonacci heap is marked. Explain how x came to be a marked root. Argue that it doesn't matter to the analysis that x is marked, even though it is not a root that was first linked to another node and then lost one child.

19.3-2

Justify the $O(1)$ amortized time of FIB-HEAP-DECREASE-KEY as an average cost per operation by using aggregate analysis.

19.4 Bounding the maximum degree

To prove that the amortized time of FIB-HEAP-EXTRACT-MIN and FIB-HEAP-DELETE is $O(\lg n)$, we must show that the upper bound $D(n)$ on the degree of any node of an n -node Fibonacci heap is $O(\lg n)$. In particular, we shall show that $D(n) \leq \lfloor \log_\phi n \rfloor$, where ϕ is the golden ratio, defined in equation (3.24) as

$$\phi = (1 + \sqrt{5})/2 = 1.61803 \dots$$

The key to the analysis is as follows. For each node x within a Fibonacci heap, define $\text{size}(x)$ to be the number of nodes, including x itself, in the subtree rooted at x . (Note that x need not be in the root list—it can be any node at all.) We shall show that $\text{size}(x)$ is exponential in $x.\text{degree}$. Bear in mind that $x.\text{degree}$ is always maintained as an accurate count of the degree of x .

Lemma 19.1

Let x be any node in a Fibonacci heap, and suppose that $x.\text{degree} = k$. Let $y_1, y_2, \dots, y_k$ denote the children of x in the order in which they were linked to x , from the earliest to the latest. Then, $y_1.\text{degree} \geq 0$ and $y_i.\text{degree} \geq i - 2$ for $i = 2, 3, \dots, k$.

Proof Obviously, $y_1.\text{degree} \geq 0$.

For $i \geq 2$, we note that when y_i was linked to x , all of $y_1, y_2, \dots, y_{i-1}$ were children of x , and so we must have had $x.\text{degree} \geq i - 1$. Because node y_i is linked to x (by CONSOLIDATE) only if $x.\text{degree} = y_i.\text{degree}$, we must have also had $y_i.\text{degree} \geq i - 1$ at that time. Since then, node y_i has lost at most one child, since it would have been cut from x (by CASCADING-CUT) if it had lost two children. We conclude that $y_i.\text{degree} \geq i - 2$. ■

We finally come to the part of the analysis that explains the name “Fibonacci heaps.” Recall from Section 3.2 that for $k = 0, 1, 2, \dots$, the k th Fibonacci number is defined by the recurrence

$$F_k = \begin{cases} 0 & \text{if } k = 0, \\ 1 & \text{if } k = 1, \\ F_{k-1} + F_{k-2} & \text{if } k \geq 2. \end{cases}$$

The following lemma gives another way to express F_k .

Lemma 19.2

For all integers $k \geq 0$,

$$F_{k+2} = 1 + \sum_{i=0}^k F_i .$$

Proof The proof is by induction on k . When $k = 0$,

$$\begin{aligned} 1 + \sum_{i=0}^0 F_i &= 1 + F_0 \\ &= 1 + 0 \\ &= F_2 . \end{aligned}$$

We now assume the inductive hypothesis that $F_{k+1} = 1 + \sum_{i=0}^{k-1} F_i$, and we have

$$\begin{aligned} F_{k+2} &= F_k + F_{k+1} \\ &= F_k + \left(1 + \sum_{i=0}^{k-1} F_i \right) \\ &= 1 + \sum_{i=0}^k F_i . \end{aligned} \quad \blacksquare$$

Lemma 19.3

For all integers $k \geq 0$, the $(k + 2)$ nd Fibonacci number satisfies $F_{k+2} \geq \phi^k$.

Proof The proof is by induction on k . The base cases are for $k = 0$ and $k = 1$. When $k = 0$ we have $F_2 = 1 = \phi^0$, and when $k = 1$ we have $F_3 = 2 > 1.619 > \phi^1$. The inductive step is for $k \geq 2$, and we assume that $F_{i+2} > \phi^i$ for $i = 0, 1, \dots, k-1$. Recall that ϕ is the positive root of equation (3.23), $x^2 = x + 1$. Thus, we have

$$\begin{aligned} F_{k+2} &= F_{k+1} + F_k \\ &\geq \phi^{k-1} + \phi^{k-2} \quad (\text{by the inductive hypothesis}) \\ &= \phi^{k-2}(\phi + 1) \\ &= \phi^{k-2} \cdot \phi^2 \quad (\text{by equation (3.23)}) \\ &= \phi^k . \end{aligned} \quad \blacksquare$$

The following lemma and its corollary complete the analysis.

Lemma 19.4

Let x be any node in a Fibonacci heap, and let $k = x.degree$. Then $size(x) \geq F_{k+2} \geq \phi^k$, where $\phi = (1 + \sqrt{5})/2$.

Proof Let s_k denote the minimum possible size of any node of degree k in any Fibonacci heap. Trivially, $s_0 = 1$ and $s_1 = 2$. The number s_k is at most $size(x)$ and, because adding children to a node cannot decrease the node's size, the value of s_k increases monotonically with k . Consider some node z , in any Fibonacci heap, such that $z.degree = k$ and $size(z) = s_k$. Because $s_k \leq size(x)$, we compute a lower bound on $size(x)$ by computing a lower bound on s_k . As in Lemma 19.1, let $y_1, y_2, \dots, y_k$ denote the children of z in the order in which they were linked to z . To bound s_k , we count one for z itself and one for the first child y_1 (for which $size(y_1) \geq 1$), giving

$$\begin{aligned} size(x) &\geq s_k \\ &\geq 2 + \sum_{i=2}^k s_{y_i.degree} \\ &\geq 2 + \sum_{i=2}^k s_{i-2}, \end{aligned}$$

where the last line follows from Lemma 19.1 (so that $y_i.degree \geq i - 2$) and the monotonicity of s_k (so that $s_{y_i.degree} \geq s_{i-2}$).

We now show by induction on k that $s_k \geq F_{k+2}$ for all nonnegative integers k . The bases, for $k = 0$ and $k = 1$, are trivial. For the inductive step, we assume that $k \geq 2$ and that $s_i \geq F_{i+2}$ for $i = 0, 1, \dots, k - 1$. We have

$$\begin{aligned} s_k &\geq 2 + \sum_{i=2}^k s_{i-2} \\ &\geq 2 + \sum_{i=2}^k F_i \\ &= 1 + \sum_{i=0}^k F_i \\ &= F_{k+2} && \text{(by Lemma 19.2)} \\ &\geq \phi^k && \text{(by Lemma 19.3)} . \end{aligned}$$

Thus, we have shown that $size(x) \geq s_k \geq F_{k+2} \geq \phi^k$. ■

Corollary 19.5

The maximum degree $D(n)$ of any node in an n -node Fibonacci heap is $O(\lg n)$.

Proof Let x be any node in an n -node Fibonacci heap, and let $k = x.\text{degree}$. By Lemma 19.4, we have $n \geq \text{size}(x) \geq \phi^k$. Taking base- ϕ logarithms gives us $k \leq \log_\phi n$. (In fact, because k is an integer, $k \leq \lfloor \log_\phi n \rfloor$.) The maximum degree $D(n)$ of any node is thus $O(\lg n)$. ■

Exercises**19.4-1**

Professor Pinocchio claims that the height of an n -node Fibonacci heap is $O(\lg n)$. Show that the professor is mistaken by exhibiting, for any positive integer n , a sequence of Fibonacci-heap operations that creates a Fibonacci heap consisting of just one tree that is a linear chain of n nodes.

19.4-2

Suppose we generalize the cascading-cut rule to cut a node x from its parent as soon as it loses its k th child, for some integer constant k . (The rule in Section 19.3 uses $k = 2$.) For what values of k is $D(n) = O(\lg n)$?

Problems
19-1 Alternative implementation of deletion

Professor Pisano has proposed the following variant of the FIB-HEAP-DELETE procedure, claiming that it runs faster when the node being deleted is not the node pointed to by $H.\text{min}$.

PISANO-DELETE(H, x)

```

1  if  $x == H.\text{min}$ 
2      FIB-HEAP-EXTRACT-MIN( $H$ )
3  else  $y = x.p$ 
4      if  $y \neq \text{NIL}$ 
5          CUT( $H, x, y$ )
6          CASCADING-CUT( $H, y$ )
7      add  $x$ 's child list to the root list of  $H$ 
8      remove  $x$  from the root list of  $H$ 
```

- a. The professor's claim that this procedure runs faster is based partly on the assumption that line 7 can be performed in $O(1)$ actual time. What is wrong with this assumption?
- b. Give a good upper bound on the actual time of PISANO-DELETE when x is not $H.min$. Your bound should be in terms of $x.degree$ and the number c of calls to the CASCADING-CUT procedure.
- c. Suppose that we call PISANO-DELETE(H, x), and let H' be the Fibonacci heap that results. Assuming that node x is not a root, bound the potential of H' in terms of $x.degree$, c , $t(H)$, and $m(H)$.
- d. Conclude that the amortized time for PISANO-DELETE is asymptotically no better than for FIB-HEAP-DELETE, even when $x \neq H.min$.

19-2 Binomial trees and binomial heaps

The **binomial tree** B_k is an ordered tree (see Section B.5.2) defined recursively. As shown in Figure 19.6(a), the binomial tree B_0 consists of a single node. The binomial tree B_k consists of two binomial trees B_{k-1} that are linked together so that the root of one is the leftmost child of the root of the other. Figure 19.6(b) shows the binomial trees B_0 through B_4 .

- a. Show that for the binomial tree B_k ,
 1. there are 2^k nodes,
 2. the height of the tree is k ,
 3. there are exactly $\binom{k}{i}$ nodes at depth i for $i = 0, 1, \dots, k$, and
 4. the root has degree k , which is greater than that of any other node; moreover, as Figure 19.6(c) shows, if we number the children of the root from left to right by $k-1, k-2, \dots, 0$, then child i is the root of a subtree B_i .

A **binomial heap** H is a set of binomial trees that satisfies the following properties:

1. Each node has a *key* (like a Fibonacci heap).
2. Each binomial tree in H obeys the min-heap property.
3. For any nonnegative integer k , there is at most one binomial tree in H whose root has degree k .
- b. Suppose that a binomial heap H has a total of n nodes. Discuss the relationship between the binomial trees that H contains and the binary representation of n . Conclude that H consists of at most $\lfloor \lg n \rfloor + 1$ binomial trees.

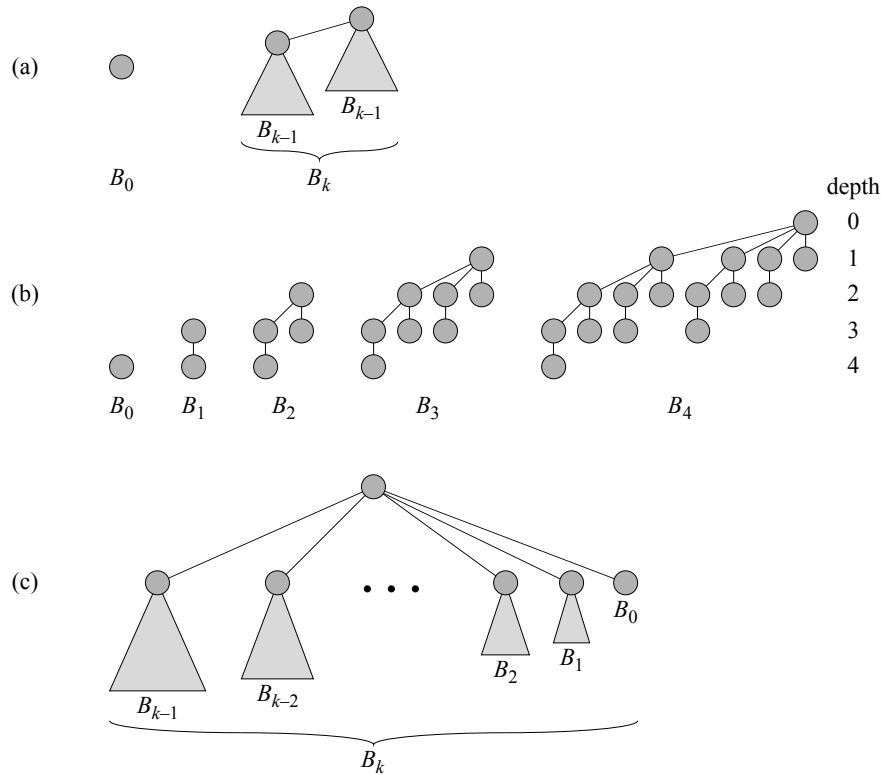


Figure 19.6 (a) The recursive definition of the binomial tree B_k . Triangles represent rooted subtrees. (b) The binomial trees B_0 through B_4 . Node depths in B_4 are shown. (c) Another way of looking at the binomial tree B_k .

Suppose that we represent a binomial heap as follows. The left-child, right-sibling scheme of Section 10.4 represents each binomial tree within a binomial heap. Each node contains its key; pointers to its parent, to its leftmost child, and to the sibling immediately to its right (these pointers are NIL when appropriate); and its degree (as in Fibonacci heaps, how many children it has). The roots form a singly linked root list, ordered by the degrees of the roots (from low to high), and we access the binomial heap by a pointer to the first node on the root list.

- c. Complete the description of how to represent a binomial heap (i.e., name the attributes, describe when attributes have the value NIL, and define how the root list is organized), and show how to implement the same seven operations on binomial heaps as this chapter implemented on Fibonacci heaps. Each operation should run in $O(\lg n)$ worst-case time, where n is the number of nodes in

the binomial heap (or in the case of the UNION operation, in the two binomial heaps that are being united). The MAKE-HEAP operation should take constant time.

- d. Suppose that we were to implement only the mergeable-heap operations on a Fibonacci heap (i.e., we do not implement the DECREASE-KEY or DELETE operations). How would the trees in a Fibonacci heap resemble those in a binomial heap? How would they differ? Show that the maximum degree in an n -node Fibonacci heap would be at most $\lfloor \lg n \rfloor$.
- e. Professor McGee has devised a new data structure based on Fibonacci heaps. A McGee heap has the same structure as a Fibonacci heap and supports just the mergeable-heap operations. The implementations of the operations are the same as for Fibonacci heaps, except that insertion and union consolidate the root list as their last step. What are the worst-case running times of operations on McGee heaps?

19-3 More Fibonacci-heap operations

We wish to augment a Fibonacci heap H to support two new operations without changing the amortized running time of any other Fibonacci-heap operations.

- a. The operation FIB-HEAP-CHANGE-KEY(H, x, k) changes the key of node x to the value k . Give an efficient implementation of FIB-HEAP-CHANGE-KEY, and analyze the amortized running time of your implementation for the cases in which k is greater than, less than, or equal to $x.key$.
- b. Give an efficient implementation of FIB-HEAP-PRUNE(H, r), which deletes $q = \min(r, H.n)$ nodes from H . You may choose any q nodes to delete. Analyze the amortized running time of your implementation. (*Hint:* You may need to modify the data structure and potential function.)

19-4 2-3-4 heaps

Chapter 18 introduced the 2-3-4 tree, in which every internal node (other than possibly the root) has two, three, or four children and all leaves have the same depth. In this problem, we shall implement **2-3-4 heaps**, which support the mergeable-heap operations.

The 2-3-4 heaps differ from 2-3-4 trees in the following ways. In 2-3-4 heaps, only leaves store keys, and each leaf x stores exactly one key in the attribute $x.key$. The keys in the leaves may appear in any order. Each internal node x contains a value $x.small$ that is equal to the smallest key stored in any leaf in the subtree rooted at x . The root r contains an attribute $r.height$ that gives the height of the

tree. Finally, 2-3-4 heaps are designed to be kept in main memory, so that disk reads and writes are not needed.

Implement the following 2-3-4 heap operations. In parts (a)–(e), each operation should run in $O(\lg n)$ time on a 2-3-4 heap with n elements. The UNION operation in part (f) should run in $O(\lg n)$ time, where n is the number of elements in the two input heaps.

- a.* MINIMUM, which returns a pointer to the leaf with the smallest key.
- b.* DECREASE-KEY, which decreases the key of a given leaf x to a given value $k \leq x.key$.
- c.* INSERT, which inserts leaf x with key k .
- d.* DELETE, which deletes a given leaf x .
- e.* EXTRACT-MIN, which extracts the leaf with the smallest key.
- f.* UNION, which unites two 2-3-4 heaps, returning a single 2-3-4 heap and destroying the input heaps.

Chapter notes

Fredman and Tarjan [114] introduced Fibonacci heaps. Their paper also describes the application of Fibonacci heaps to the problems of single-source shortest paths, all-pairs shortest paths, weighted bipartite matching, and the minimum-spanning-tree problem.

Subsequently, Driscoll, Gabow, Shrairman, and Tarjan [96] developed “relaxed heaps” as an alternative to Fibonacci heaps. They devised two varieties of relaxed heaps. One gives the same amortized time bounds as Fibonacci heaps. The other allows DECREASE-KEY to run in $O(1)$ worst-case (not amortized) time and EXTRACT-MIN and DELETE to run in $O(\lg n)$ worst-case time. Relaxed heaps also have some advantages over Fibonacci heaps in parallel algorithms.

See also the chapter notes for Chapter 6 for other data structures that support fast DECREASE-KEY operations when the sequence of values returned by EXTRACT-MIN calls are monotonically increasing over time and the data are integers in a specific range.

In previous chapters, we saw data structures that support the operations of a priority queue—binary heaps in Chapter 6, red-black trees in Chapter 13,¹ and Fibonacci heaps in Chapter 19. In each of these data structures, at least one important operation took $O(\lg n)$ time, either worst case or amortized. In fact, because each of these data structures bases its decisions on comparing keys, the $\Omega(n \lg n)$ lower bound for sorting in Section 8.1 tells us that at least one operation will have to take $\Omega(\lg n)$ time. Why? If we could perform both the INSERT and EXTRACT-MIN operations in $o(\lg n)$ time, then we could sort n keys in $o(n \lg n)$ time by first performing n INSERT operations, followed by n EXTRACT-MIN operations.

We saw in Chapter 8, however, that sometimes we can exploit additional information about the keys to sort in $o(n \lg n)$ time. In particular, with counting sort we can sort n keys, each an integer in the range 0 to k , in time $\Theta(n + k)$, which is $\Theta(n)$ when $k = O(n)$.

Since we can circumvent the $\Omega(n \lg n)$ lower bound for sorting when the keys are integers in a bounded range, you might wonder whether we can perform each of the priority-queue operations in $o(\lg n)$ time in a similar scenario. In this chapter, we shall see that we can: van Emde Boas trees support the priority-queue operations, and a few others, each in $O(\lg \lg n)$ worst-case time. The hitch is that the keys must be integers in the range 0 to $n - 1$, with no duplicates allowed.

Specifically, van Emde Boas trees support each of the dynamic set operations listed on page 230—SEARCH, INSERT, DELETE, MINIMUM, MAXIMUM, SUCCESSOR, and PREDECESSOR—in $O(\lg \lg n)$ time. In this chapter, we will omit discussion of satellite data and focus only on storing keys. Because we concentrate on keys and disallow duplicate keys to be stored, instead of describing the SEARCH

¹Chapter 13 does not explicitly discuss how to implement EXTRACT-MIN and DECREASE-KEY, but we can easily build these operations for any data structure that supports MINIMUM, DELETE, and INSERT.

operation, we will implement the simpler operation $\text{MEMBER}(S, x)$, which returns a boolean indicating whether the value x is currently in dynamic set S .

So far, we have used the parameter n for two distinct purposes: the number of elements in the dynamic set, and the range of the possible values. To avoid any further confusion, from here on we will use n to denote the number of elements currently in the set and u as the range of possible values, so that each van Emde Boas tree operation runs in $O(\lg \lg u)$ time. We call the set $\{0, 1, 2, \dots, u-1\}$ the *universe* of values that can be stored and u the *universe size*. We assume throughout this chapter that u is an exact power of 2, i.e., $u = 2^k$ for some integer $k \geq 1$.

Section 20.1 starts us out by examining some simple approaches that will get us going in the right direction. We enhance these approaches in Section 20.2, introducing proto van Emde Boas structures, which are recursive but do not achieve our goal of $O(\lg \lg u)$ -time operations. Section 20.3 modifies proto van Emde Boas structures to develop van Emde Boas trees, and it shows how to implement each operation in $O(\lg \lg u)$ time.

20.1 Preliminary approaches

In this section, we shall examine various approaches for storing a dynamic set. Although none will achieve the $O(\lg \lg u)$ time bounds that we desire, we will gain insights that will help us understand van Emde Boas trees when we see them later in this chapter.

Direct addressing

Direct addressing, as we saw in Section 11.1, provides the simplest approach to storing a dynamic set. Since in this chapter we are concerned only with storing keys, we can simplify the direct-addressing approach to store the dynamic set as a bit vector, as discussed in Exercise 11.1-2. To store a dynamic set of values from the universe $\{0, 1, 2, \dots, u-1\}$, we maintain an array $A[0..u-1]$ of u bits. The entry $A[x]$ holds a 1 if the value x is in the dynamic set, and it holds a 0 otherwise. Although we can perform each of the INSERT, DELETE, and MEMBER operations in $O(1)$ time with a bit vector, the remaining operations—MINIMUM, MAXIMUM, SUCCESSOR, and PREDECESSOR—each take $\Theta(u)$ time in the worst case because

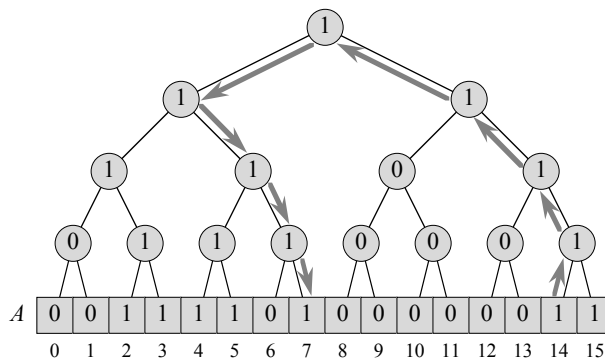


Figure 20.1 A binary tree of bits superimposed on top of a bit vector representing the set $\{2, 3, 4, 5, 7, 14, 15\}$ when $u = 16$. Each internal node contains a 1 if and only if some leaf in its subtree contains a 1. The arrows show the path followed to determine the predecessor of 14 in the set.

we might have to scan through $\Theta(u)$ elements.² For example, if a set contains only the values 0 and $u - 1$, then to find the successor of 0, we would have to scan entries 1 through $u - 2$ before finding a 1 in $A[u - 1]$.

Superimposing a binary tree structure

We can short-cut long scans in the bit vector by superimposing a binary tree of bits on top of it. Figure 20.1 shows an example. The entries of the bit vector form the leaves of the binary tree, and each internal node contains a 1 if and only if any leaf in its subtree contains a 1. In other words, the bit stored in an internal node is the logical-or of its two children.

The operations that took $\Theta(u)$ worst-case time with an unadorned bit vector now use the tree structure:

- To find the minimum value in the set, start at the root and head down toward the leaves, always taking the leftmost node containing a 1.
- To find the maximum value in the set, start at the root and head down toward the leaves, always taking the rightmost node containing a 1.

²We assume throughout this chapter that MINIMUM and MAXIMUM return NIL if the dynamic set is empty and that SUCCESSOR and PREDECESSOR return NIL if the element they are given has no successor or predecessor, respectively.

- To find the successor of x , start at the leaf indexed by x , and head up toward the root until we enter a node from the left and this node has a 1 in its right child z . Then head down through node z , always taking the leftmost node containing a 1 (i.e., find the minimum value in the subtree rooted at the right child z).
- To find the predecessor of x , start at the leaf indexed by x , and head up toward the root until we enter a node from the right and this node has a 1 in its left child z . Then head down through node z , always taking the rightmost node containing a 1 (i.e., find the maximum value in the subtree rooted at the left child z).

Figure 20.1 shows the path taken to find the predecessor, 7, of the value 14.

We also augment the INSERT and DELETE operations appropriately. When inserting a value, we store a 1 in each node on the simple path from the appropriate leaf up to the root. When deleting a value, we go from the appropriate leaf up to the root, recomputing the bit in each internal node on the path as the logical-or of its two children.

Since the height of the tree is $\lg u$ and each of the above operations makes at most one pass up the tree and at most one pass down, each operation takes $O(\lg u)$ time in the worst case.

This approach is only marginally better than just using a red-black tree. We can still perform the MEMBER operation in $O(1)$ time, whereas searching a red-black tree takes $O(\lg n)$ time. Then again, if the number n of elements stored is much smaller than the size u of the universe, a red-black tree would be faster for all the other operations.

Superimposing a tree of constant height

What happens if we superimpose a tree with greater degree? Let us assume that the size of the universe is $u = 2^{2^k}$ for some integer k , so that $\sqrt{u}$ is an integer. Instead of superimposing a binary tree on top of the bit vector, we superimpose a tree of degree $\sqrt{u}$. Figure 20.2(a) shows such a tree for the same bit vector as in Figure 20.1. The height of the resulting tree is always 2.

As before, each internal node stores the logical-or of the bits within its subtree, so that the $\sqrt{u}$ internal nodes at depth 1 summarize each group of $\sqrt{u}$ values. As Figure 20.2(b) demonstrates, we can think of these nodes as an array *summary*[0.. $\sqrt{u}-1$], where *summary*[i] contains a 1 if and only if the subarray $A[i\sqrt{u}..(i+1)\sqrt{u}-1]$ contains a 1. We call this $\sqrt{u}$ -bit subarray of A the i th **cluster**. For a given value of x , the bit $A[x]$ appears in cluster number $\lfloor x/\sqrt{u} \rfloor$. Now INSERT becomes an $O(1)$ -time operation: to insert x , set both $A[x]$ and *summary*[$\lfloor x/\sqrt{u} \rfloor$] to 1. We can use the *summary* array to perform

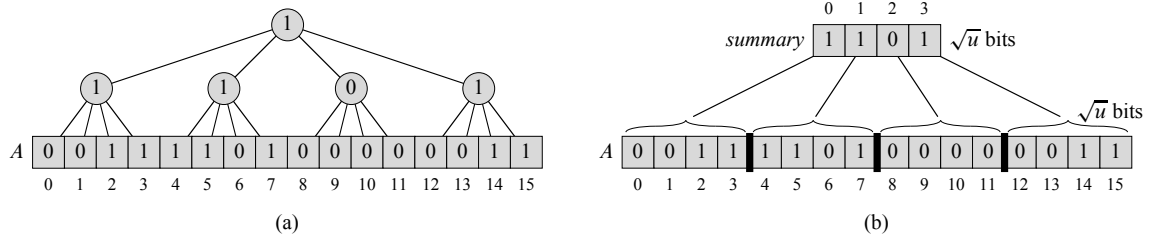


Figure 20.2 (a) A tree of degree $\sqrt{u}$ superimposed on top of the same bit vector as in Figure 20.1. Each internal node stores the logical-or of the bits in its subtree. (b) A view of the same structure, but with the internal nodes at depth 1 treated as an array $summary[0.. \sqrt{u} - 1]$, where $summary[i]$ is the logical-or of the subarray $A[i\sqrt{u}.. (i+1)\sqrt{u} - 1]$.

each of the operations MINIMUM, MAXIMUM, SUCCESSOR, PREDECESSOR, and DELETE in $O(\sqrt{u})$ time:

- To find the minimum (maximum) value, find the leftmost (rightmost) entry in $summary$ that contains a 1, say $summary[i]$, and then do a linear search within the i th cluster for the leftmost (rightmost) 1.
- To find the successor (predecessor) of x , first search to the right (left) within its cluster. If we find a 1, that position gives the result. Otherwise, let $i = \lfloor x/\sqrt{u} \rfloor$ and search to the right (left) within the $summary$ array from index i . The first position that holds a 1 gives the index of a cluster. Search within that cluster for the leftmost (rightmost) 1. That position holds the successor (predecessor).
- To delete the value x , let $i = \lfloor x/\sqrt{u} \rfloor$. Set $A[x]$ to 0 and then set $summary[i]$ to the logical-or of the bits in the i th cluster.

In each of the above operations, we search through at most two clusters of $\sqrt{u}$ bits plus the $summary$ array, and so each operation takes $O(\sqrt{u})$ time.

At first glance, it seems as though we have made negative progress. Superimposing a binary tree gave us $O(\lg u)$ -time operations, which are asymptotically faster than $O(\sqrt{u})$ time. Using a tree of degree $\sqrt{u}$ will turn out to be a key idea of van Emde Boas trees, however. We continue down this path in the next section.

Exercises

20.1-1

Modify the data structures in this section to support duplicate keys.

20.1-2

Modify the data structures in this section to support keys that have associated satellite data.

20.1-3

Observe that, using the structures in this section, the way we find the successor and predecessor of a value x does not depend on whether x is in the set at the time. Show how to find the successor of x in a binary search tree when x is not stored in the tree.

20.1-4

Suppose that instead of superimposing a tree of degree $\sqrt{u}$, we were to superimpose a tree of degree $u^{1/k}$, where $k > 1$ is a constant. What would be the height of such a tree, and how long would each of the operations take?

20.2 A recursive structure

In this section, we modify the idea of superimposing a tree of degree $\sqrt{u}$ on top of a bit vector. In the previous section, we used a summary structure of size $\sqrt{u}$, with each entry pointing to another structure of size $\sqrt{u}$. Now, we make the structure recursive, shrinking the universe size by the square root at each level of recursion. Starting with a universe of size u , we make structures holding $\sqrt{u} = u^{1/2}$ items, which themselves hold structures of $u^{1/4}$ items, which hold structures of $u^{1/8}$ items, and so on, down to a base size of 2.

For simplicity, in this section, we assume that $u = 2^{2^k}$ for some integer k , so that $u, u^{1/2}, u^{1/4}, \dots$ are integers. This restriction would be quite severe in practice, allowing only values of u in the sequence 2, 4, 16, 256, 65536, $\dots$. We shall see in the next section how to relax this assumption and assume only that $u = 2^k$ for some integer k . Since the structure we examine in this section is only a precursor to the true van Emde Boas tree structure, we tolerate this restriction in favor of aiding our understanding.

Recalling that our goal is to achieve running times of $O(\lg \lg u)$ for the operations, let's think about how we might obtain such running times. At the end of Section 4.3, we saw that by changing variables, we could show that the recurrence

$$T(n) = 2T(\lfloor \sqrt{n} \rfloor) + \lg n \quad (20.1)$$

has the solution $T(n) = O(\lg n \lg \lg n)$. Let's consider a similar, but simpler, recurrence:

$$T(u) = T(\sqrt{u}) + O(1). \quad (20.2)$$

If we use the same technique, changing variables, we can show that recurrence (20.2) has the solution $T(u) = O(\lg \lg u)$. Let $m = \lg u$, so that $u = 2^m$ and we have

$$T(2^m) = T(2^{m/2}) + O(1) .$$

Now we rename $S(m) = T(2^m)$, giving the new recurrence

$$S(m) = S(m/2) + O(1) .$$

By case 2 of the master method, this recurrence has the solution $S(m) = O(\lg m)$. We change back from $S(m)$ to $T(u)$, giving $T(u) = T(2^m) = S(m) = O(\lg m) = O(\lg \lg u)$.

Recurrence (20.2) will guide our search for a data structure. We will design a recursive data structure that shrinks by a factor of $\sqrt{u}$ in each level of its recursion. When an operation traverses this data structure, it will spend a constant amount of time at each level before recursing to the level below. Recurrence (20.2) will then characterize the running time of the operation.

Here is another way to think of how the term $\lg \lg u$ ends up in the solution to recurrence (20.2). As we look at the universe size in each level of the recursive data structure, we see the sequence $u, u^{1/2}, u^{1/4}, u^{1/8}, \dots$. If we consider how many bits we need to store the universe size at each level, we need $\lg u$ at the top level, and each level needs half the bits of the previous level. In general, if we start with b bits and halve the number of bits at each level, then after $\lg b$ levels, we get down to just one bit. Since $b = \lg u$, we see that after $\lg \lg u$ levels, we have a universe size of 2.

Looking back at the data structure in Figure 20.2, a given value x resides in cluster number $\lfloor x/\sqrt{u} \rfloor$. If we view x as a $\lg u$ -bit binary integer, that cluster number, $\lfloor x/\sqrt{u} \rfloor$, is given by the most significant $(\lg u)/2$ bits of x . Within its cluster, x appears in position $x \bmod \sqrt{u}$, which is given by the least significant $(\lg u)/2$ bits of x . We will need to index in this way, and so let us define some functions that will help us do so:

$$\begin{aligned} \text{high}(x) &= \lfloor x/\sqrt{u} \rfloor , \\ \text{low}(x) &= x \bmod \sqrt{u} , \\ \text{index}(x, y) &= x\sqrt{u} + y . \end{aligned}$$

The function $\text{high}(x)$ gives the most significant $(\lg u)/2$ bits of x , producing the number of x 's cluster. The function $\text{low}(x)$ gives the least significant $(\lg u)/2$ bits of x and provides x 's position within its cluster. The function $\text{index}(x, y)$ builds an element number from x and y , treating x as the most significant $(\lg u)/2$ bits of the element number and y as the least significant $(\lg u)/2$ bits. We have the identity $x = \text{index}(\text{high}(x), \text{low}(x))$. The value of u used by each of these functions will

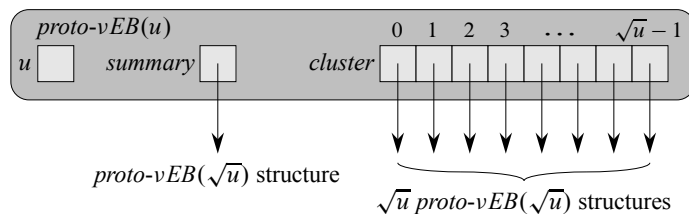


Figure 20.3 The information in a $\text{proto-vEB}(u)$ structure when $u \geq 4$. The structure contains the universe size u , a pointer summary to a $\text{proto-vEB}(\sqrt{u})$ structure, and an array $\text{cluster}[0 \dots \sqrt{u}-1]$ of $\sqrt{u}$ pointers to $\text{proto-vEB}(\sqrt{u})$ structures.

always be the universe size of the data structure in which we call the function, which changes as we descend into the recursive structure.

20.2.1 Proto van Emde Boas structures

Taking our cue from recurrence (20.2), let us design a recursive data structure to support the operations. Although this data structure will fail to achieve our goal of $O(\lg \lg u)$ time for some operations, it serves as a basis for the van Emde Boas tree structure that we will see in Section 20.3.

For the universe $\{0, 1, 2, \dots, u-1\}$, we define a **proto van Emde Boas structure**, or **proto-vEB structure**, which we denote as $\text{proto-vEB}(u)$, recursively as follows. Each $\text{proto-vEB}(u)$ structure contains an attribute u giving its universe size. In addition, it contains the following:

- If $u = 2$, then it is the base size, and it contains an array $A[0 \dots 1]$ of two bits.
- Otherwise, $u = 2^{2^k}$ for some integer $k \geq 1$, so that $u \geq 4$. In addition to the universe size u , the data structure $\text{proto-vEB}(u)$ contains the following attributes, illustrated in Figure 20.3:
 - a pointer named summary to a $\text{proto-vEB}(\sqrt{u})$ structure and
 - an array $\text{cluster}[0 \dots \sqrt{u}-1]$ of $\sqrt{u}$ pointers, each to a $\text{proto-vEB}(\sqrt{u})$ structure.

The element x , where $0 \leq x < u$, is recursively stored in the cluster numbered $\text{high}(x)$ as element $\text{low}(x)$ within that cluster.

In the two-level structure of the previous section, each node stores a summary array of size $\sqrt{u}$, in which each entry contains a bit. From the index of each entry, we can compute the starting index of the subarray of size $\sqrt{u}$ that the bit summarizes. In the proto-vEB structure, we use explicit pointers rather than index

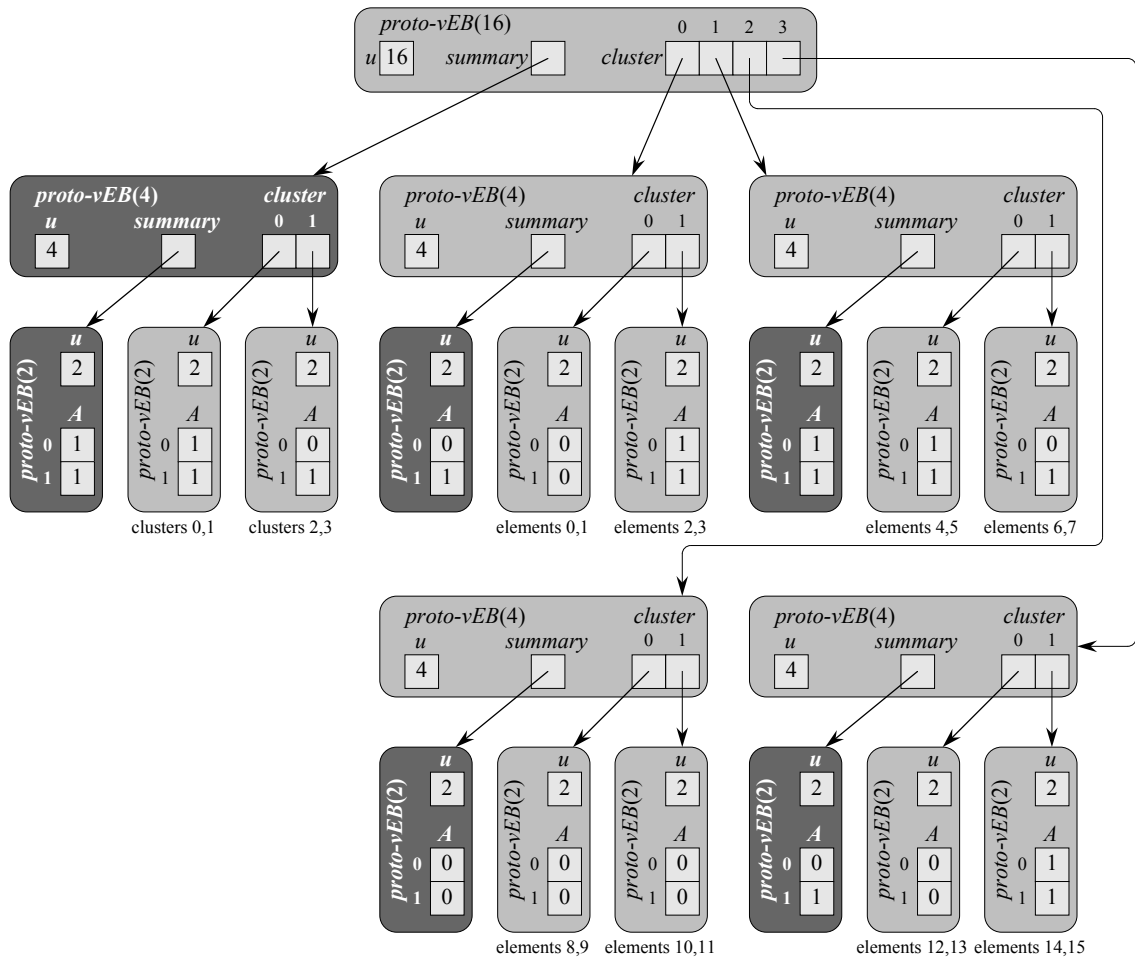


Figure 20.4 A *proto-vEB(16)* structure representing the set $\{2, 3, 4, 5, 7, 14, 15\}$. It points to four *proto-vEB(4)* structures in *cluster*[0..3], and to a summary structure, which is also a *proto-vEB(4)*. Each *proto-vEB(4)* structure points to two *proto-vEB(2)* structures in *cluster*[0..1], and to a *proto-vEB(2)* summary. Each *proto-vEB(2)* structure contains just an array *A*[0..1] of two bits. The *proto-vEB(2)* structures above “elements *i, j*” store bits *i* and *j* of the actual dynamic set, and the *proto-vEB(2)* structures above “clusters *i, j*” store the summary bits for clusters *i* and *j* in the top-level *proto-vEB(16)* structure. For clarity, heavy shading indicates the top level of a *proto-vEB* structure that stores summary information for its parent structure; such a *proto-vEB* structure is otherwise identical to any other *proto-vEB* structure with the same universe size.

calculations. The array *summary* contains the summary bits stored recursively in a proto-vEB structure, and the array *cluster* contains $\sqrt{u}$ pointers.

Figure 20.4 shows a fully expanded *proto-vEB*(16) structure representing the set $\{2, 3, 4, 5, 7, 14, 15\}$. If the value i is in the proto-vEB structure pointed to by *summary*, then the i th cluster contains some value in the set being represented. As in the tree of constant height, *cluster*[i] represents the values $i\sqrt{u}$ through $(i + 1)\sqrt{u} - 1$, which form the i th cluster.

At the base level, the elements of the actual dynamic sets are stored in some of the *proto-vEB*(2) structures, and the remaining *proto-vEB*(2) structures store summary bits. Beneath each of the non-summary base structures, the figure indicates which bits it stores. For example, the *proto-vEB*(2) structure labeled “elements 6,7” stores bit 6 (0, since element 6 is not in the set) in its $A[0]$ and bit 7 (1, since element 7 is in the set) in its $A[1]$.

Like the clusters, each summary is just a dynamic set with universe size $\sqrt{u}$, and so we represent each summary as a *proto-vEB*($\sqrt{u}$) structure. The four summary bits for the main *proto-vEB*(16) structure are in the leftmost *proto-vEB*(4) structure, and they ultimately appear in two *proto-vEB*(2) structures. For example, the *proto-vEB*(2) structure labeled “clusters 2,3” has $A[0] = 0$, indicating that cluster 2 of the *proto-vEB*(16) structure (containing elements 8, 9, 10, 11) is all 0, and $A[1] = 1$, telling us that cluster 3 (containing elements 12, 13, 14, 15) has at least one 1. Each *proto-vEB*(4) structure points to its own summary, which is itself stored as a *proto-vEB*(2) structure. For example, look at the *proto-vEB*(2) structure just to the left of the one labeled “elements 0,1.” Because its $A[0]$ is 0, it tells us that the “elements 0,1” structure is all 0, and because its $A[1]$ is 1, we know that the “elements 2,3” structure contains at least one 1.

20.2.2 Operations on a proto van Emde Boas structure

We shall now describe how to perform operations on a proto-vEB structure. We first examine the query operations—MEMBER, MINIMUM, and SUCCESSOR—which do not change the proto-vEB structure. We then discuss INSERT and DELETE. We leave MAXIMUM and PREDECESSOR, which are symmetric to MINIMUM and SUCCESSOR, respectively, as Exercise 20.2-1.

Each of the MEMBER, SUCCESSOR, PREDECESSOR, INSERT, and DELETE operations takes a parameter x , along with a proto-vEB structure V . Each of these operations assumes that $0 \leq x < V.u$.

Determining whether a value is in the set

To perform MEMBER(x), we need to find the bit corresponding to x within the appropriate *proto-vEB*(2) structure. We can do so in $O(\lg \lg u)$ time, bypassing

the *summary* structures altogether. The following procedure takes a *proto-vEB* structure V and a value x , and it returns a bit indicating whether x is in the dynamic set held by V .

```

PROTO-VEB-MEMBER( $V, x$ )
1  if  $V.u == 2$ 
2      return  $V.A[x]$ 
3  else return PROTO-VEB-MEMBER( $V.cluster[high(x)], low(x)$ )

```

The PROTO-VEB-MEMBER procedure works as follows. Line 1 tests whether we are in a base case, where V is a *proto-vEB*(2) structure. Line 2 handles the base case, simply returning the appropriate bit of array A . Line 3 deals with the recursive case, “drilling down” into the appropriate smaller *proto-vEB* structure. The value $high(x)$ says which *proto-vEB*($\sqrt{u}$) structure we visit, and $low(x)$ determines which element within that *proto-vEB*($\sqrt{u}$) structure we are querying.

Let’s see what happens when we call PROTO-VEB-MEMBER($V, 6$) on the *proto-vEB*(16) structure in Figure 20.4. Since $high(6) = 1$ when $u = 16$, we recurse into the *proto-vEB*(4) structure in the upper right, and we ask about element $low(6) = 2$ of that structure. In this recursive call, $u = 4$, and so we recurse again. With $u = 4$, we have $high(2) = 1$ and $low(2) = 0$, and so we ask about element 0 of the *proto-vEB*(2) structure in the upper right. This recursive call turns out to be a base case, and so it returns $A[0] = 0$ back up through the chain of recursive calls. Thus, we get the result that PROTO-VEB-MEMBER($V, 6$) returns 0, indicating that 6 is not in the set.

To determine the running time of PROTO-VEB-MEMBER, let $T(u)$ denote its running time on a *proto-vEB*(u) structure. Each recursive call takes constant time, not including the time taken by the recursive calls that it makes. When PROTO-VEB-MEMBER makes a recursive call, it makes a call on a *proto-vEB*($\sqrt{u}$) structure. Thus, we can characterize the running time by the recurrence $T(u) = T(\sqrt{u}) + O(1)$, which we have already seen as recurrence (20.2). Its solution is $T(u) = O(\lg \lg u)$, and so we conclude that PROTO-VEB-MEMBER runs in time $O(\lg \lg u)$.

Finding the minimum element

Now we examine how to perform the MINIMUM operation. The procedure PROTO-VEB-MINIMUM(V) returns the minimum element in the *proto-vEB* structure V , or NIL if V represents an empty set.

PROTO-VEB-MINIMUM(V)

```

1  if  $V.u == 2$ 
2      if  $V.A[0] == 1$ 
3          return 0
4      elseif  $V.A[1] == 1$ 
5          return 1
6      else return NIL
7  else  $min\_cluster = \text{PROTO-VEB-MINIMUM}(V.summary)$ 
8      if  $min\_cluster == \text{NIL}$ 
9          return NIL
10     else  $offset = \text{PROTO-VEB-MINIMUM}(V.cluster[min\_cluster])$ 
11     return  $\text{index}(min\_cluster, offset)$ 

```

This procedure works as follows. Line 1 tests for the base case, which lines 2–6 handle by brute force. Lines 7–11 handle the recursive case. First, line 7 finds the number of the first cluster that contains an element of the set. It does so by recursively calling PROTO-VEB-MINIMUM on $V.summary$, which is a *proto-veb*($\sqrt{u}$) structure. Line 7 assigns this cluster number to the variable *min-cluster*. If the set is empty, then the recursive call returned NIL, and line 9 returns NIL. Otherwise, the minimum element of the set is somewhere in cluster number *min-cluster*. The recursive call in line 10 finds the offset within the cluster of the minimum element in this cluster. Finally, line 11 constructs the value of the minimum element from the cluster number and offset, and it returns this value.

Although querying the summary information allows us to quickly find the cluster containing the minimum element, because this procedure makes two recursive calls on *proto-veb*($\sqrt{u}$) structures, it does not run in $O(\lg \lg u)$ time in the worst case. Letting $T(u)$ denote the worst-case time for PROTO-VEB-MINIMUM on a *proto-veb*(u) structure, we have the recurrence

$$T(u) = 2T(\sqrt{u}) + O(1). \quad (20.3)$$

Again, we use a change of variables to solve this recurrence, letting $m = \lg u$, which gives

$$T(2^m) = 2T(2^{m/2}) + O(1).$$

Renaming $S(m) = T(2^m)$ gives

$$S(m) = 2S(m/2) + O(1),$$

which, by case 1 of the master method, has the solution $S(m) = \Theta(m)$. By changing back from $S(m)$ to $T(u)$, we have that $T(u) = T(2^m) = S(m) = \Theta(m) = \Theta(\lg u)$. Thus, we see that because of the second recursive call, PROTO-VEB-MINIMUM runs in $\Theta(\lg u)$ time rather than the desired $O(\lg \lg u)$ time.

Finding the successor

The SUCCESSOR operation is even worse. In the worst case, it makes two recursive calls, along with a call to PROTO-VEB-MINIMUM. The procedure PROTO-VEB-SUCCESSOR(V, x) returns the smallest element in the proto-vEB structure V that is greater than x , or NIL if no element in V is greater than x . It does not require x to be a member of the set, but it does assume that $0 \leq x < V.u$.

PROTO-VEB-SUCCESSOR(V, x)

```

1  if  $V.u == 2$ 
2      if  $x == 0$  and  $V.A[1] == 1$ 
3          return 1
4      else return NIL
5  else  $offset = \text{PROTO-VEB-SUCCESSOR}(V.cluster[high(x)], low(x))$ 
6      if  $offset \neq \text{NIL}$ 
7          return  $\text{index}(high(x), offset)$ 
8      else  $succ-cluster = \text{PROTO-VEB-SUCCESSOR}(V.summary, high(x))$ 
9          if  $succ-cluster == \text{NIL}$ 
10             return NIL
11         else  $offset = \text{PROTO-VEB-MINIMUM}(V.cluster[succ-cluster])$ 
12             return  $\text{index}(succ-cluster, offset)$ 
```

The PROTO-VEB-SUCCESSOR procedure works as follows. As usual, line 1 tests for the base case, which lines 2–4 handle by brute force: the only way that x can have a successor within a *proto-vEB*(2) structure is when $x = 0$ and $A[1]$ is 1. Lines 5–12 handle the recursive case. Line 5 searches for a successor to x within x 's cluster, assigning the result to *offset*. Line 6 determines whether x has a successor within its cluster; if it does, then line 7 computes and returns the value of this successor. Otherwise, we have to search in other clusters. Line 8 assigns to *succ-cluster* the number of the next nonempty cluster, using the summary information to find it. Line 9 tests whether *succ-cluster* is NIL, with line 10 returning NIL if all succeeding clusters are empty. If *succ-cluster* is non-NIL, line 11 assigns the first element within that cluster to *offset*, and line 12 computes and returns the minimum element in that cluster.

In the worst case, PROTO-VEB-SUCCESSOR calls itself recursively twice on *proto-vEB*($\sqrt{u}$) structures, and it makes one call to PROTO-VEB-MINIMUM on a *proto-vEB*($\sqrt{u}$) structure. Thus, the recurrence for the worst-case running time $T(u)$ of PROTO-VEB-SUCCESSOR is

$$\begin{aligned}
 T(u) &= 2T(\sqrt{u}) + \Theta(\lg \sqrt{u}) \\
 &= 2T(\sqrt{u}) + \Theta(\lg u) .
 \end{aligned}$$

We can employ the same technique that we used for recurrence (20.1) to show that this recurrence has the solution $T(u) = \Theta(\lg u \lg \lg u)$. Thus, PROTO-VEB-SUCCESSOR is asymptotically slower than PROTO-VEB-MINIMUM.

Inserting an element

To insert an element, we need to insert it into the appropriate cluster and also set the summary bit for that cluster to 1. The procedure PROTO-VEB-INSERT(V, x) inserts the value x into the proto-vEB structure V .

PROTO-VEB-INSERT(V, x)

```

1  if  $V.u == 2$ 
2       $V.A[x] = 1$ 
3  else PROTO-VEB-INSERT( $V.cluster[high(x)], low(x)$ )
4      PROTO-VEB-INSERT( $V.summary, high(x)$ )
```

In the base case, line 2 sets the appropriate bit in the array A to 1. In the recursive case, the recursive call in line 3 inserts x into the appropriate cluster, and line 4 sets the summary bit for that cluster to 1.

Because PROTO-VEB-INSERT makes two recursive calls in the worst case, recurrence (20.3) characterizes its running time. Hence, PROTO-VEB-INSERT runs in $\Theta(\lg u)$ time.

Deleting an element

The DELETE operation is more complicated than insertion. Whereas we can always set a summary bit to 1 when inserting, we cannot always reset the same summary bit to 0 when deleting. We need to determine whether any bit in the appropriate cluster is 1. As we have defined proto-vEB structures, we would have to examine all $\sqrt{u}$ bits within a cluster to determine whether any of them are 1. Alternatively, we could add an attribute n to the proto-vEB structure, counting how many elements it has. We leave implementation of PROTO-VEB-DELETE as Exercises 20.2-2 and 20.2-3.

Clearly, we need to modify the proto-vEB structure to get each operation down to making at most one recursive call. We will see in the next section how to do so.

Exercises

20.2-1

Write pseudocode for the procedures PROTO-VEB-MAXIMUM and PROTO-VEB-PREDECESSOR.

20.2-2

Write pseudocode for `PROTO-VEB-DELETE`. It should update the appropriate summary bit by scanning the related bits within the cluster. What is the worst-case running time of your procedure?

20.2-3

Add the attribute n to each proto-veB structure, giving the number of elements currently in the set it represents, and write pseudocode for `PROTO-VEB-DELETE` that uses the attribute n to decide when to reset summary bits to 0. What is the worst-case running time of your procedure? What other procedures need to change because of the new attribute? Do these changes affect their running times?

20.2-4

Modify the proto-veB structure to support duplicate keys.

20.2-5

Modify the proto-veB structure to support keys that have associated satellite data.

20.2-6

Write pseudocode for a procedure that creates a *proto-veB*(u) structure.

20.2-7

Argue that if line 9 of `PROTO-VEB-MINIMUM` is executed, then the proto-veB structure is empty.

20.2-8

Suppose that we designed a proto-veB structure in which each *cluster* array had only $u^{1/4}$ elements. What would the running times of each operation be?

20.3 The van Emde Boas tree

The proto-veB structure of the previous section is close to what we need to achieve $O(\lg \lg u)$ running times. It falls short because we have to recurse too many times in most of the operations. In this section, we shall design a data structure that is similar to the proto-veB structure but stores a little more information, thereby removing the need for some of the recursion.

In Section 20.2, we observed that the assumption that we made about the universe size—that $u = 2^{2^k}$ for some integer k —is unduly restrictive, confining the possible values of u to an overly sparse set. From this point on, therefore, we will allow the universe size u to be any exact power of 2, and when $\sqrt{u}$ is not an inte-

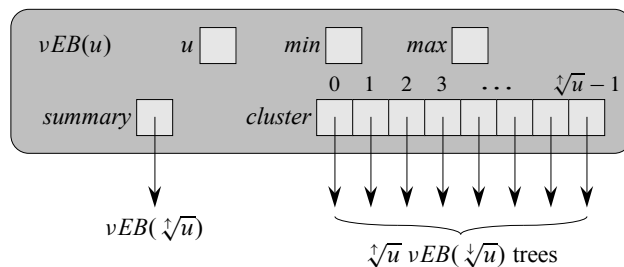


Figure 20.5 The information in a $vEB(u)$ tree when $u > 2$. The structure contains the universe size u , elements min and max , a pointer $summary$ to a $vEB(\sqrt[4]{u})$ tree, and an array $cluster[0.. \sqrt[4]{u} - 1]$ of $\sqrt[4]{u}$ pointers to $vEB(\sqrt[4]{u})$ trees.

ger—that is, if u is an odd power of 2 ($u = 2^{2k+1}$ for some integer $k \geq 0$)—then we will divide the $\lg u$ bits of a number into the most significant $\lceil (\lg u)/2 \rceil$ bits and the least significant $\lfloor (\lg u)/2 \rfloor$ bits. For convenience, we denote $2^{\lceil (\lg u)/2 \rceil}$ (the “upper square root” of u) by $\sqrt[4]{u}$ and $2^{\lfloor (\lg u)/2 \rfloor}$ (the “lower square root” of u) by $\sqrt[4]{u}$, so that $u = \sqrt[4]{u} \cdot \sqrt[4]{u}$ and, when u is an even power of 2 ($u = 2^{2k}$ for some integer k), $\sqrt[4]{u} = \sqrt[4]{u} = \sqrt{u}$. Because we now allow u to be an odd power of 2, we must redefine our helpful functions from Section 20.2:

$$\begin{aligned} \text{high}(x) &= \lfloor x / \sqrt[4]{u} \rfloor, \\ \text{low}(x) &= x \bmod \sqrt[4]{u}, \\ \text{index}(x, y) &= x \sqrt[4]{u} + y. \end{aligned}$$

20.3.1 van Emde Boas trees

The *van Emde Boas tree*, or *vEB tree*, modifies the proto-vEB structure. We denote a vEB tree with a universe size of u as $vEB(u)$ and, unless u equals the base size of 2, the attribute $summary$ points to a $vEB(\sqrt[4]{u})$ tree and the array $cluster[0.. \sqrt[4]{u} - 1]$ points to $\sqrt[4]{u}$ $vEB(\sqrt[4]{u})$ trees. As Figure 20.5 illustrates, a vEB tree contains two attributes not found in a proto-vEB structure:

- min stores the minimum element in the vEB tree, and
- max stores the maximum element in the vEB tree.

Furthermore, the element stored in min does not appear in any of the recursive $vEB(\sqrt[4]{u})$ trees that the $cluster$ array points to. The elements stored in a $vEB(u)$ tree V , therefore, are $V.min$ plus all the elements recursively stored in the $vEB(\sqrt[4]{u})$ trees pointed to by $V.cluster[0.. \sqrt[4]{u} - 1]$. Note that when a vEB tree contains two or more elements, we treat min and max differently: the element

stored in *min* does not appear in any of the clusters, but unless the vEB tree contains just one element (so that the minimum and maximum elements are the same), the element stored in *max* does.

Since the base size is 2, a $vEB(2)$ tree does not need the array A that the corresponding *proto-vEB(2)* structure has. Instead, we can determine its elements from its *min* and *max* attributes. In a vEB tree with no elements, regardless of its universe size u , both *min* and *max* are NIL.

Figure 20.6 shows a $vEB(16)$ tree V holding the set $\{2, 3, 4, 5, 7, 14, 15\}$. Because the smallest element is 2, $V.min$ equals 2, and even though $high(2) = 0$, the element 2 does not appear in the $vEB(4)$ tree pointed to by $V.cluster[0]$: notice that $V.cluster[0].min$ equals 3, and so 2 is not in this vEB tree. Similarly, since $V.cluster[0].min$ equals 3, and 2 and 3 are the only elements in $V.cluster[0]$, the $vEB(2)$ clusters within $V.cluster[0]$ are empty.

The *min* and *max* attributes will turn out to be key to reducing the number of recursive calls within the operations on vEB trees. These attributes will help us in four ways:

1. The MINIMUM and MAXIMUM operations do not even need to recurse, for they can just return the values of *min* or *max*.
2. The SUCCESSOR operation can avoid making a recursive call to determine whether the successor of a value x lies within $high(x)$. That is because x 's successor lies within its cluster if and only if x is strictly less than the *max* attribute of its cluster. A symmetric argument holds for PREDECESSOR and *min*.
3. We can tell whether a vEB tree has no elements, exactly one element, or at least two elements in constant time from its *min* and *max* values. This ability will help in the INSERT and DELETE operations. If *min* and *max* are both NIL, then the vEB tree has no elements. If *min* and *max* are non-NIL but are equal to each other, then the vEB tree has exactly one element. Otherwise, both *min* and *max* are non-NIL but are unequal, and the vEB tree has two or more elements.
4. If we know that a vEB tree is empty, we can insert an element into it by updating only its *min* and *max* attributes. Hence, we can insert into an empty vEB tree in constant time. Similarly, if we know that a vEB tree has only one element, we can delete that element in constant time by updating only *min* and *max*. These properties will allow us to cut short the chain of recursive calls.

Even if the universe size u is an odd power of 2, the difference in the sizes of the summary vEB tree and the clusters will not turn out to affect the asymptotic running times of the vEB-tree operations. The recursive procedures that implement the vEB-tree operations will all have running times characterized by the recurrence

$$T(u) \leq T(\sqrt[4]{u}) + O(1). \quad (20.4)$$

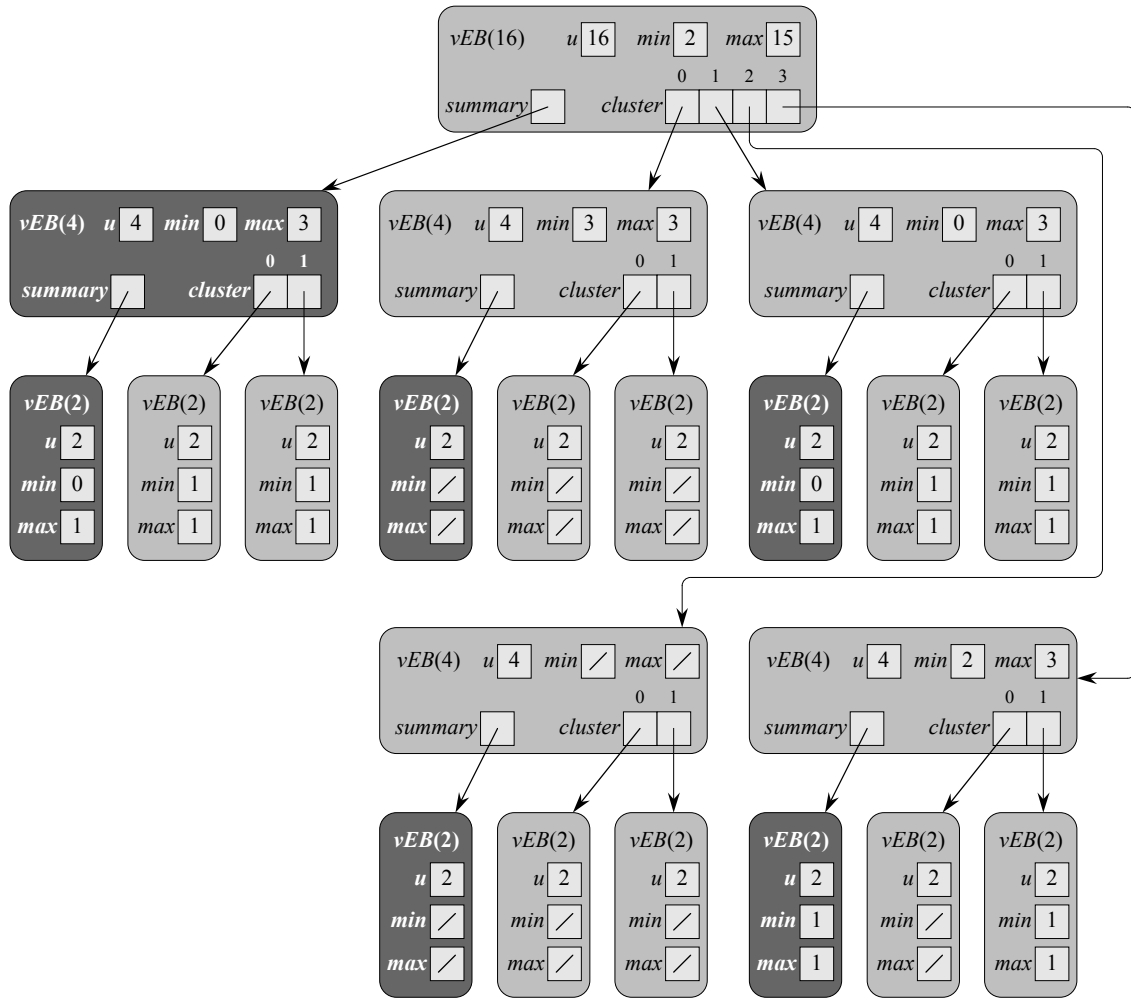


Figure 20.6 A $vEB(16)$ tree corresponding to the proto- vEB tree in Figure 20.4. It stores the set $\{2, 3, 4, 5, 7, 14, 15\}$. Slashes indicate NIL values. The value stored in the min attribute of a vEB tree does not appear in any of its clusters. Heavy shading serves the same purpose here as in Figure 20.4.

This recurrence looks similar to recurrence (20.2), and we will solve it in a similar fashion. Letting $m = \lg u$, we rewrite it as

$$T(2^m) \leq T(2^{\lceil m/2 \rceil}) + O(1) .$$

Noting that $\lceil m/2 \rceil \leq 2m/3$ for all $m \geq 2$, we have

$$T(2^m) \leq T(2^{2m/3}) + O(1) .$$

Letting $S(m) = T(2^m)$, we rewrite this last recurrence as

$$S(m) \leq S(2m/3) + O(1) ,$$

which, by case 2 of the master method, has the solution $S(m) = O(\lg m)$. (In terms of the asymptotic solution, the fraction $2/3$ does not make any difference compared with the fraction $1/2$, because when we apply the master method, we find that $\log_{3/2} 1 = \log_2 1 = 0$.) Thus, we have $T(u) = T(2^m) = S(m) = O(\lg m) = O(\lg \lg u)$.

Before using a van Emde Boas tree, we must know the universe size u , so that we can create a van Emde Boas tree of the appropriate size that initially represents an empty set. As Problem 20-1 asks you to show, the total space requirement of a van Emde Boas tree is $O(u)$, and it is straightforward to create an empty tree in $O(u)$ time. In contrast, we can create an empty red-black tree in constant time. Therefore, we might not want to use a van Emde Boas tree when we perform only a small number of operations, since the time to create the data structure would exceed the time saved in the individual operations. This drawback is usually not significant, since we typically use a simple data structure, such as an array or linked list, to represent a set with only a few elements.

20.3.2 Operations on a van Emde Boas tree

We are now ready to see how to perform operations on a van Emde Boas tree. As we did for the proto van Emde Boas structure, we will consider the querying operations first, and then INSERT and DELETE. Due to the slight asymmetry between the minimum and maximum elements in a vEB tree—when a vEB tree contains at least two elements, the minimum element does not appear within a cluster but the maximum element does—we will provide pseudocode for all five querying operations. As in the operations on proto van Emde Boas structures, the operations here that take parameters V and x , where V is a van Emde Boas tree and x is an element, assume that $0 \leq x < V.u$.

Finding the minimum and maximum elements

Because we store the minimum and maximum in the attributes *min* and *max*, two of the operations are one-liners, taking constant time: